

LINKED LIST

1.

a) What is linked list? What are the advantages and disadvantages of linked list over the linear array?

Ans: A linked list, or one-way list, is a linear collection of data elements, called nodes, where the linear order is maintained by means of links or pointers.

Advantages of a Linked List over a Linear Array:

- **Dynamic Size:** Linked lists can grow or shrink during runtime by adding or removing nodes, unlike arrays which have a fixed size (in static implementations like C).
- **Efficient Insertions/Deletions:** Adding or removing elements (especially at the beginning or middle) is faster in a linked list ($O(1)$ for head insertion) because it only requires adjusting pointers, whereas arrays require shifting elements ($O(n)$).
- **No Wasted Space:** Memory is allocated only for the elements present, unlike arrays where unused allocated space can be wasted.

Disadvantages of a Linked List over a Linear Array:

- **No Random Access:** Linked lists don't support direct access to elements by index; you must traverse from the head to reach the desired node ($O(n)$ time), while arrays allow $O(1)$ access via indexing.
- **Extra Memory Overhead:** Each node in a linked list requires additional memory for storing pointers, whereas arrays store only the data.
- **Poor Cache Performance:** Since nodes are not stored contiguously in memory, linked lists suffer from poor spatial locality, leading to more cache misses compared to arrays, which benefit from contiguous storage.

b) Write an algorithm to search a linked list when the list is sorted.

Ans:

```
SRCHSL(INFO, LINK, START, ITEM, LOC)
LIST is a sorted list in memory. This algorithm finds the location LOC of the
node where ITEM first appears in LIST, or sets LOC := NULL.
1. Set PTR := START.
2. Repeat Step 3 while PTR ≠ NULL:
3.   If ITEM < INFO[PTR], then:
       Set PTR := LINK[PTR]. [PTR now points to next node.]
   Else if ITEM = INFO[PTR], then:
       Set LOC := PTR, and Exit. [Search is successful.]
   Else:
       Set LOC := NULL, and Exit. [ITEM now exceeds INFO[PTR].]
   [End of If structure.]
   [End of Step 2 loop.]
4. Set LOC := NULL.
5. Exit.
```

LINKED LIST

- c) Consider the *alphabetized* list of patients in Figure 1. Determine the changes in the data structure if
- Jones is added to the list and then
 - Max is deleted from the list.

START = 5; AVAIL = 10

BED	1	2	3	4	5	6	7	8	9	10	11	12
Name	Kirk		Dean	Max	Adam		Lane	Green	Samu		Fields	Nelso
Link	7	6	11	12	3	0	4	1	0	2	8	9

Figure 1

Ans: Alphabetized list based on names.

- START = 5 (Adam)
- AVAIL = 10
- BEDs 2, 6, 10 are free (no names), and available for use.

The list from BED 5 onward (following links) is: Adam (5) → Dean (3) → Fields (11) → Green (8) → Kirk (1) → Lane (7) → Max (4) → Nelso (12) → Samu (9)

i) If Jones is added to the list:

- Alphabetically, **Jones** comes between **Green (8)** and **Kirk (1)**.
- Use BED **10** from AVAIL for Jones.
- Update:
 - Link[8] = 10 → Jones inserted after Green
 - Link[10] = 1 → Jones points to Kirk
 - AVAIL = 2 → next available BED

Updated list:

BED	1	2	3	4	5	6	7	8	9	10	11	12
Name	Kirk		Dean	Max	Adam		Lane	Green	Samu	Jones	Fields	Nelso
Link	7	6	11	12	3	0	4	10	0	1	8	9

ii) If Max is deleted from the list:

- Max is at **BED 4**, its previous node is **Lane (7)**
- Link[7] = 12 (Lane will now point to Nelso instead of Max)
- Add Max's BED (4) back to the free list:
 - Link[4] = AVAIL → Link[4] = 2
 - AVAIL = 4

LINKED LIST

Final structure after both operations:

BED	1	2	3	4	5	6	7	8	9	10	11	12
Name	Kirk		Dean		Adam		Lane	Green	Samu	Jones	Fields	Nelso
Link	7	6	11	2	3	0	12	10	0	1	8	9

d) What is two way lists? Form a two way list from the one way list in Figure 1.

Ans: A two-way list, also known as a doubly linked list, is a linked list where each node contains data and two pointers: one to the next node and one to the previous node. This allows traversal in both forward and backward directions, unlike a singly linked list (one-way list) which only has a next pointer.

Two-Way List from Figure 1 (before insert/delete for simplicity):

BED	Name	Prev	Next
5	Adam	NULL	8
8	Green	5	1
1	Kirk	8	3
3	Dean	1	4
4	Max	3	7
7	Lane	4	9
9	Samu	7	11
11	Fields	9	12
12	Nelso	11	NULL

LINKED LIST

2.

a) What is **circular header list**? Write the **advantages of circular header list** over ordinary linked list.

Ans: A **Circular Header List** is a special type of **circular linked list** that includes a **header node** at the beginning. The list is circular, meaning the last node points back to the **header node**, and not to NULL. The header node doesn't store actual data — it typically stores metadata (like count, or is just a placeholder).

☒ **Advantages of Circular Header List over Ordinary Linked List:**

Feature	Circular Header List	Ordinary Linked List
No NULL needed	No node points to NULL; easier traversal	Traversal must check for NULL
Simpler insertion/deletion	Operations (especially at ends) are more uniform	Edge cases for head/tail need special handling
Always a known starting point	Header node simplifies algorithms	May need extra logic to maintain start
Can represent empty list easily	Header points to itself when empty	head = NULL and needs checking
Efficient circular traversal	Good for scheduling, buffering, etc.	Not naturally suited for circular tasks

LINKED LIST

b) You are given a linked list containing an **odd number of elements**. You don't know how many numbers there are in that list. Write an algorithm or C/C++ code to **find the middle element** of this list.

Ans:

MIDSL(HEAD, MID)

This algorithm finds the middle node of a singly linked list.

1. Set SLOW := HEAD
2. Set FAST := HEAD
3. Repeat **while** FAST $\neq$ NULL and LINK[FAST] $\neq$ NULL:
 Set SLOW := LINK[SLOW] [Move SLOW one step]
 Set FAST := LINK[LINK[FAST]] [Move FAST two steps]
 [End of loop]
4. Set MID := SLOW [SLOW now points to the middle node]
5. Exit.

```
struct Node {
    int data;
    Node* next;
};

// Function to find the middle of a singly linked list
void findMiddle(Node* head) {
    Node* slow = head;
    Node* fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    if (slow != NULL)
        cout << "Middle element: " << slow->data << endl;
}
```

LINKED LIST

OR

b) You are given a **two-way linked list** containing an odd number of elements. You don't know how many numbers there are in that list. Write an algorithm or C/C++ code to **find the middle element of this list by iterating only once** over the list.

Ans:

MIDDL(DHEAD, MID)

This algorithm finds the middle node of a doubly linked list using a single traversal.

1. Set SLOW := DHEAD
2. Set FAST := DHEAD
3. Repeat while FAST ≠ NULL and LINK[FAST] ≠ NULL:
 Set SLOW := LINK[SLOW] [Move SLOW one step]
 Set FAST := LINK[LINK[FAST]] [Move FAST two steps]
 [End of loop]
4. Set MID := SLOW [SLOW now points to the middle node]
5. Exit.

```
struct DNode {
    int data;
    DNode* prev;
    DNode* next;
};

// Function to find middle of doubly linked list
void findMiddle(DNode* head) {
    DNode* slow = head;
    DNode* fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    if (slow != NULL)
        cout << "Middle element: " << slow->data << endl;
}
```

LINKED LIST

d) A DEQUE is a data structure that allows insertion and deletion of elements from both ends that allows the following four operations on it:

1. **PUSH_FRONT(X)**: This function adds an item X in front of its elements.
2. **PUSH_BACK(X)**: This function adds an item X at the end of its elements.
3. **POP_FRONT()**: This function deletes the first element (if there is any).
4. **POP_BACK()**: This function deletes the last element (if there is any).

Suppose you are building a **DEQUE** where the underlying data structure would be a **one way linked list**. What would be the **time complexity** of these four operations? Explain.

Ans:

1. PUSH_FRONT(X)

Operation: Insert at the beginning

Time Complexity: $O(1)$

Explanation:

- We can insert a node directly at the front by updating the head pointer.
- No traversal is needed.

2. PUSH_BACK(X)

Operation: Insert at the end

Time Complexity: $O(n)$

Explanation:

- In a singly linked list, to insert at the end, we must **traverse the list to the last node** (since there's no tail pointer).
- Hence, it takes linear time.

3. POP_FRONT()

Operation: Delete from the beginning

Time Complexity: $O(1)$

Explanation:

- We can delete the front node by updating the head pointer to the next node.
- Constant time operation.

LINKED LIST

4. POP_BACK()

Operation: Delete from the end

Time Complexity: $O(n)$

Explanation:

- To delete the last node, we must **traverse the list to find the second-last node**, then set its link to NULL.
- This requires traversing all nodes, so it's linear time.

Summary Table

Operation	Time Complexity	Reason
PUSH_FRONT(X)	$O(1)$	Directly add at head
PUSH_BACK(X)	$O(n)$	Need to traverse to the end
POP_FRONT()	$O(1)$	Directly remove from head
POP_BACK()	$O(n)$	Need to traverse to second last node

 **Note:** In the previous mid-semester exam, **Stack** was included in the syllabus, but for the upcoming mid, it has been **replaced by Linked List**. Since the exact pattern of Linked List questions is uncertain, I have solved relevant problems from **previous final exam papers** to ensure coverage of commonly asked concepts.